CCTV LIVE HEAD COUNTER & CROWD ANALYTICS

System Architecture, Model Benchmarking, Sensitivity Tuning & Indian Footage Validation

Project: AI CCTV Head Counter **Author:** Ikshit Joshi (Calance) **Date:** November 2026 **Status:** Production Ready

1. Executive Summary

This system implements an edge-accelerated, high-accuracy **CCTV Live Head Counter and Crowd Surveillance Engine**. Standard face detectors fail in overhead CCTV environments because people look down or walk away. Full-body pedestrian detectors fail in crowded spaces because human bodies occlude each other by >70%. By leveraging specialized single-class YOLO models trained on overhead/oblique human head contours, this system delivers real-time detection, persistent multi-object tracking, bidirectional line crossing (tripwire IN/OUT), and polygonal Region of Interest (ROI) density estimation at 20–45+ FPS with Apple Silicon (MPS) and NVIDIA (CUDA) GPU acceleration.

Key Deliverables Completed:

- **Full Decoupled Architecture:** Clean separation of Detector, Tracker, Geometry Analytics, and MJPEG Streamer.
- **7 YOLO Models Supported:** Nano, Small, and Medium variants across YOLO11 and YOLOv8 benchmarked.
- **Dense Indian CCTV Footage Added:** Validated on Kolkata Flower Market and Howrah Concourse (up to 65 heads/frame).
- **Empirically Validated Sweet Spot:** Confidence threshold of **0.20 – 0.25** verified as optimal for crowded recall vs. precision.
- **Operator Dashboard:** Full surveillance web UI with dynamic model switching, RTSP ingest, and 1-click CSV export.

2. System Architecture & File Responsibilities

File / Module	Core Functionality	Algorithmic / Architectural Details
config.py	Master System Config	Single source of truth for RTSP URLs, server ports, model choices, confidence thresholds, tripwire coordinates, and HUD colors.
engine/detector.py	YOLO Head Detector	Auto-detects Apple Silicon MPS, NVIDIA CUDA, or CPU. Loads YOLO checkpoints and extracts bounding boxes, confidences, and centerpoints.
engine/tracker.py	Persistent Object Tracker	Bipartite matching via Euclidean distance + IoU cost scoring. Maintains persistent track IDs, motion trails, and purges lost tracks after 15 frames.
engine/counter.py	Spatial & Direction Analytics	Computes: 1) Instantaneous in-frame count; 2) ROI count via **Ray-Casting**; 3) Directional tripwire IN/OUT count via **Line-Intersection & 2D Cross-Product**.
engine/pipeline.py	Unified Video Pipeline	Orchestrates video capture, runs detector -> tracker -> counter, renders CCTV overlays (boxes, trails, crosshairs, HUD), and yields MJPEG streams.
app.py	FastAPI Web Server	Exposes /video_feed (MJPEG), /api/status, /api/history, source switching, video upload, tripwire/ROI config, and CSV export.
templates/index.html	Surveillance Dashboard	Dark-mode operator UI with live video stream, channel selector, model switcher, sensitivity slider, interactive zones, and real-time Chart.js graph.
cli.py	Standalone CLI Tool	Runs head counting from terminal without browser. Supports live OpenCV rendering window (--show) and batch offline video rendering (--save).
download_models.py	Weights Downloader	Automated utility downloading all 7 pretrained YOLO head models from HuggingFace into models/ with progress reporting.

3. AI Detection Models Benchmarking

We benchmarked **7 YOLO models** across YOLO11 and YOLOv8 architectures to balance edge throughput and crowd capacity:

Model Name	Weights	Latency (MPS)	Dense Crowd Recall	Precision Profile	Best Deployment Fit
yolo11n-head	5.2 MB	22 ms (~45 FPS)	Moderate (31 heads)	Balanced	Edge streaming default
yolo11s-head	18.3 MB	28 ms (~35 FPS)	**Highest (65 heads)**	High confidence	**Recommended for dense crowds**
yolo11m-head	38.6 MB	42 ms (~24 FPS)	High (50 heads)	Robust features	Severe occlusions & shadows
yolov8n-head	6.0 MB	11 ms (~90 FPS)	Moderate (29 heads)	Zero false-alarm bias	Ultra-low-power CPU / IoT
yolov8s-head	21.5 MB	24 ms (~40 FPS)	High (50 heads)	Zero false boxes	High-precision turnstiles
yolov8m-head	52.0 MB	46 ms (~22 FPS)	High (57 heads)	Highest avg conf (0.67)	Wide-angle transit hubs
yolo11n-person	5.6 MB	20 ms (~50 FPS)	Fails in dense crowds	Full-body dependent	Sparse doorway tracking only

4. Validation on Dense Indian CCTV Scenarios

To rigorously stress-test detection under complex real-world conditions, two authentic Indian surveillance channels were integrated:

Channel 04: Kolkata Flower Market (Mallick Ghat)

• *Context & Environment:* Asia's largest wholesale flower market under Howrah Bridge. High pedestrian friction, vendors carrying large baskets on heads, overhead tarps, and shadowed corridors.
• *Surveillance Accuracy:* Detects **10 to 18 active heads per frame**. The model successfully distinguished between woven flower baskets and human heads without misclassifications.

Channel 05: Howrah Concourse Crowd Assembly

• *Context & Environment:* Ultra-dense public street gathering in Howrah, West Bengal with shoulder-to-shoulder movement and >60% body occlusions.
• *Surveillance Accuracy:* In test Frame 200, yolo11s-head successfully detected **65 simultaneous heads**, proving robust capability in high-density Indian public spaces.

Empirical Performance Benchmark on Howrah Dense Crowd (Frame 200)

Model Evaluated	Heads Detected	Avg Confidence	Crowd Occlusion Assessment
yolo11n-head (Nano)	31 heads	0.59	Captures foreground and mid-ground heads; misses smaller background heads.
yolo11s-head (Small)	**65 heads**	**0.60**	**Highest overall recall**; resolves deeply occluded heads in dense clusters.
yolo11m-head (Medium)	50 heads	0.61	Very tight bounding boxes; conservative on boundary/edge heads.
yolov8s-head (Small)	50 heads	0.62	Clean detections; completely eliminates background clothing artifacts.
yolov8m-head (Medium)	57 heads	0.67	Highest confidence scores; strong performance across high-contrast shadows.

5. Technical & Operational Management Q&A

A. What is the sweet spot threshold for detection sensitivity, and why?

The optimal sensitivity threshold is **0.20 to 0.25 (20% – 25%)** for crowded surveillance and **0.35 to 0.40** for entrance turnstiles. The reasoning is grounded in empirical confidence sweeps on the Howrah crowd footage:

Confidence Threshold	Heads Detected	Operational Trade-off & Accuracy Profile
0.10 – 0.15	73 – 79 heads	**False Positive Risk:** High sensitivity; hallucinates heads on backpacks, sacks, and shoulder seams.
0.20 – 0.25 (Sweet Spot)	**65 – 69 heads**	**Optimal Balance:** Detects distant/partially occluded heads (raw conf ~0.22–0.32) while eliminating noise.
0.35 – 0.45	44 – 52 heads	**High Precision:** Zero false alarms; ideal for entrance gates where pedestrians pass individually.
0.50 – 0.60+	29 – 43 heads	**Severe Undercounting:** Misses >40% of heads due to downwards tilt, caps, turbans, or ambient shadows.

B. Which models are we using, and why?

• **Head vs. Face Models:** Face detectors (MTCNN, RetinaFace) require eyes/nose/mouth and fail in 70% of overhead CCTV views. Head detectors identify the cranium, hair, caps, and neck boundary from all angles (overhead, rear, profile).
• **Head vs. Full-Body Models:** In dense gatherings, human bodies are >70% occluded by other people. Only human heads remain consistently visible in the top visual plane.
• **YOLO11 vs. YOLOv8:** YOLOv8 provides strict precision (zero false alarms on empty scenes), while YOLO11 provides higher recall for dense overlapping objects.

C. Can these models be fine-tuned? What weights are they using?

• **Weights Origin:** Pretrained on COCO backbones, then fine-tuned on **~15,000 annotated surveillance images** (SCUT-HEAD and CrowdHuman) specifically for class 0 (head).
• **Can Calance fine-tune them further? Yes, 100%.** Because these are native PyTorch .pt Ultralytics checkpoints, the team can fine-tune them on company-specific CCTV cameras with just 200–500 labeled frames to adapt to specific mounting heights or IR night vision:

```
Python Fine-Tuning Script:
from ultralytics import YOLO
model = YOLO('models/11s-head.pt') # Start from existing head checkpoint
model.train(data='custom_cctv.yaml', epochs=30, imgsz=640, batch=16, lr0=0.001)
```

6. Deployment Recommendations & Enterprise Roadmap

1. **Production Configuration:** Deploy yolo11s-head at confidence 0.25 with IoU 0.45 for the best balance of speed and crowd recall.
2. **Live Ingestion:** Connect to enterprise IP cameras/NVRs via RTSP stream URLs in config.py (e.g. rtsp://admin:pass@ip:554/stream1).
3. **Multi-Camera Scaling:** Containerize via Docker and use NVIDIA DeepStream/TensorRT FP16 to run 8–16 simultaneous camera streams on a single GPU.
4. **BI Analytics Integration:** Use /api/export_csv to ingest hourly headcount, peak, and net flow metrics into corporate dashboards.

Report Prepared By: Ikshit Joshi | AI & Surveillance Systems **Organization:** Calance **Review Status:** Production Ready